Methoden verwenden

Einführung in die Programmierung 1
TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Zusicherungen
- Vor- und Nachbedingungen
- assert-Anweisung in Java

Was wird in diesem Foliensatz besprochen?

- Java API
- Methoden implementieren
- Schrittweises Verfeinern
- Refactoring
- Debugging von Methoden

Was lernen Sie kennen?

- Ausgewählte Klassen aus der Java-API
- Importieren von Klassen
- Implementierung von Methoden
- Schrittweise Programme verbessern
- Refactoring von Programmen
- Debugging bei Methoden

Java-API (erster Ausblick)

Java bietet etliche statische Methoden (Klassenmethoden) in speziellen Klassen an

- Gehören nur zur Klasse, nie zu einem Objekt

Typischer Aufruf

- ClassName.MethodName(Parameter)
- Beispiel
  - Math.sqrt(10.0);

Manchmal auch in der Form

- ClassName.ObjectName.MethodName(Parameter)
- Beispiel
  - System.out.println(x);

Beispiele (1)

Beispiele für Klasse **System**
• System.currentTimeMillis() • Statische Variable out   • System.out.println()

Beispiele für Klasse **Math**
• Math.*random*() • Math.*sqrt*(10.0) • …

Beispiele (2)

Byte, Short, Integer, Float, Double, Character

• Klassen für primitive Datentypen
  • Methoden, Konstanten
• Beispiele für Klasse Integer
  • Integer.max(2,3)
  • Integer.*parseInt*(**…**)
  • …

Importieren – Allgemein

Bestimmte Klassen werden automatisch geladen

- Z. B. Integer, Math, System

Andere Klassen müssen importiert werden

Importieren – 2 Arten

Expliziter Import (am Anfang der Datei)

```
import java.util.Scanner;
import java.util.*;
```

Impliziter Import (bei Verwendung)

```
java.util.Arrays.sort(x);
```

Pakete

Große Programme beinhalten viele Klassen

- Probleme: Orientierung, Namenskonflikte, …

Paket

- Sammlung mehrerer logisch zusammengehörender Klassen

Regeln für Pakete

In einem Paket sollte eine Klasse eindeutig benannt werden

In unterschiedlichen Paketen kollidieren nicht gleiche Namen

Pakete werden benannt (wie Variablen etc.)

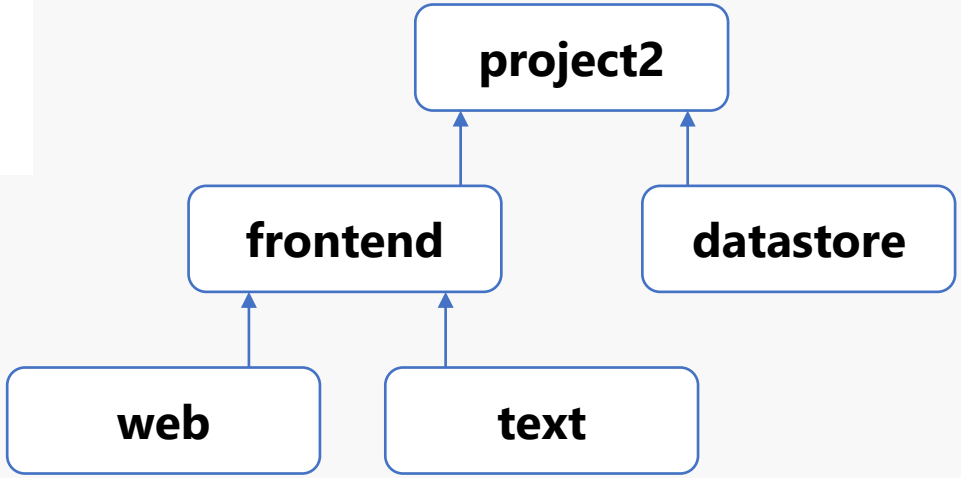
Pakete können geschachtelt werden

- Es gibt immer höchstens ein Super-Package
- Pakete bilden eine Baumstruktur

Beispiel (Pakete)

Beispiel für Pakete

```
    project2
    project2.frontend
    project2.frontend.web
    project2.frontend.text
    project2.datastore
```



Module

- Seit Java 9
- Weitere Stufe über den Paketen
- Ermöglicht bessere Modularisierung (Aufteilung größerer Programme)
- Mehr dazu in späteren Semestern

Dokumentation von Methoden

- Startpunkt (Java 25)
  - https://docs.oracle.com/en/java/javase/25/docs/api/index.html

Dokumentation – Informationen

In welchem Modul und Paket finden wir die Klasse?

Für eine Klasse

- Beschreibung der Klasse
- Liste der Methoden

Für jede Methode

- Kurzbeschreibung
- Längere Beschreibung (Parameter, Ausnahmen etc.)

DRY (Don‘t Repeat Yourself)-Prinzip

Duplizierten Code vermeiden – führt sonst zu

- Umfangreicherem Code
- Mehr Aufwand bei Änderungen

In Methoden auslagern (Beispiele)

- Komplizierte Bedingungen
- Umfangreiche Berechnungen
- Sequenz von Anweisungen (Algorithmen)

Kleine Methoden bevorzugen

Daumenregel

- Bis zu 20 Zeilen
- Wenn möglich, aber kleiner
- Methode muss aber noch sinnvoll sein

Einrückungstiefe bzw. Verschachtelungstiefe in einer Methode

- Ein bis zwei Ebenen

Anzahl der Argumente

Möglichst wenige Parameter verwenden

Größere Anzahl der Parameter macht das Aufrufen komplizierter

- Reihenfolge der Argumente
- Wichtigkeit der einzelnen Argumente

Bei mehr Argumenten

- Einige Argumente in eine separate Klasse einhüllen (wird noch bei der Objektorientierung besprochen)

Flag-Argumente vermeiden

Boolean-Flag als Argument

- Schlechter Stil
- Zeigt an, dass eine Methode mehr als eine Aufgabe erfüllt (nämlich zwei)

Methode in zwei Methoden aufspalten

- Eine Methode für den true-Zweig
- Eine Methode für den false-Zweig

Seiteneffekte vermeiden

Seiteneffekt

- Methode verspricht eine Aufgabe zu erfüllen, aber erledigt auch andere verborgene Aufgaben
- Zeitliche Kopplung von Ereignissen, die möglicherweise gar nicht gleichzeitig passieren sollten

Beispiele

- Ausgabe auf den Bildschirm (oder in eine Datei), obwohl die Methode nur eine Berechnung durchführen sollte
- Unerwartete Änderungen von Variablen einer Klasse

Anweisung und Abfrage trennen

Methode sollte

- etwas tun **oder**
- etwas zurückliefern

Mischung führt zu Verwirrung

Beispiel

```
public boolean set (String attribute, String value)
```

…

- Wert eines Attributs wird gesetzt und true zurückgeliefert, wenn dies erfolgreich ist, und false, wenn kein solches Attribut existiert

Anwendungsfall

```
if (set("username", "bob")) …
```

Ist diese Zeile aussagekräftig? Was wird hier gemacht?

Beispiel – bessere Variante

Bessere Variante mit zwei Methoden

- Eine überprüft, ob das Attribut existiert (z. B. `attributeExists`)
- Eine zweite setzt das Attribut auf einen neuen Wert (z. B. `setAttribute`)

```
if (attributeExists("username")) {
    setAttribute("username", "bob");
}
```

Schrittweises Verfeinern

Allgemeine Reihe von Schritten zur Lösung des Problems angeben (jeden Schritt einzeln definieren)

Das Problem in eine Reihe kleinerer Teilschritte unterteilen usw.

Aufhören, wenn einzelner Schritt sich mit einer Methode sinnvoll implementieren lässt

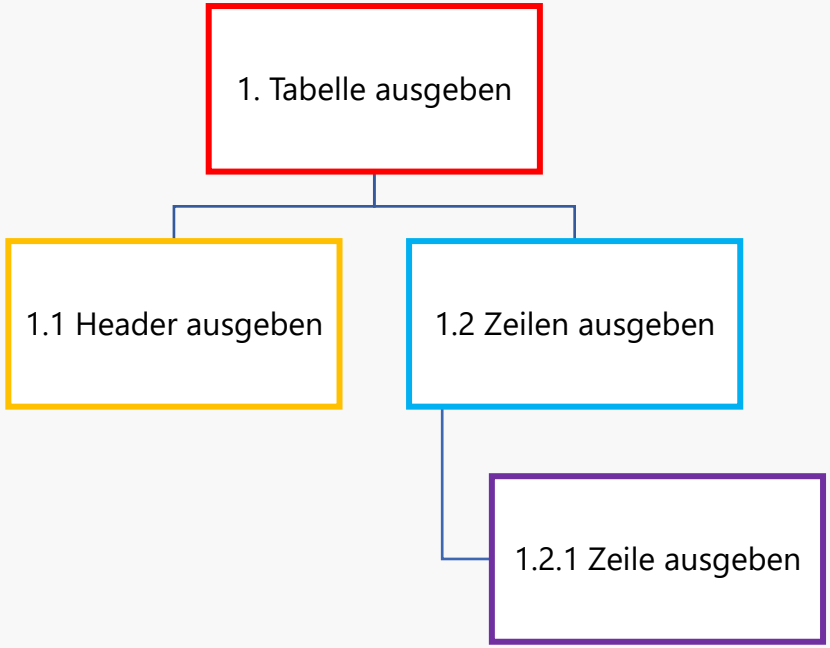
Schrittweises Verfeinern – kleines Beispiel

```java
public class Multiplication {
    public static void main(String[] args) {
        int n = 5;
        System.out.print("*\t");
        for (int i = 1; i <= n; i++) {
            System.out.print("\t" + i);
        }
        System.out.println("\n");
        for (int i = 1; i <= n; i++) {
            System.out.print(i + "\t");
            for (int j = 1; j <= n; j++) {
                System.out.print("\t" + i * j);
            }
            System.out.println();
        }
    }
}
```

Multiplikationstabelle – Originalcode (alles in der main-Methode)

Multiplikationstabelle – mögliche Schritte

```java
public class Multiplication {
    public static void main(String[] args) {
        int n = 5;
        System.out.print("*\t");
        for (int i = 1; i <= n; i++) {
            System.out.print("\t" + i);
        }
        System.out.println("\n");
        for (int i = 1; i <= n; i++) {
            System.out.print(i + "\t");
            for (int j = 1; j <= n; j++) {
                System.out.print("\t" + i * j);
            }
            System.out.println();
        }
    }
}
```



Multiplikationstabelle neu

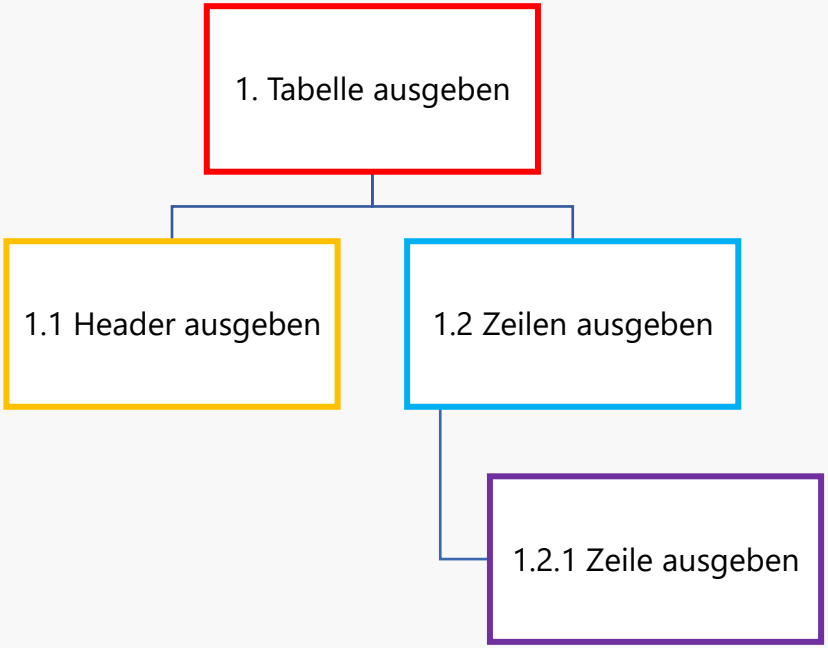
```java
public class Multiplication {
    public static void main(String[] args) {
        int n = 5;
        printMultiplicationTable(n);
    }

    private static void printMultiplicationTable(int n) {
        printMultiplicationTableHeader(n);
        printMultiplicationTableRows(n);
    }

    private static void printMultiplicationTableHeader(int n) {
        System.out.print("*\t");
        for (int i = 1; i <= n; i++) {
            System.out.print("\t" + i);
        }
        System.out.println("\n");
    }

    private static void printMultiplicationTableRows(int n) {
        for (int i = 1; i <= n; i++) {
            printMultiplicationTableRow(i, n);
        }
    }

    private static void printMultiplicationTableRow(int row, int n) {
        System.out.print(row + "\t");
        for (int j = 1; j <= n; j++) {
            System.out.print("\t" + row * j);
        }
        System.out.println();
    }
}
```



Automatisch Methodenköpfe generieren

Aufruf einer Methode implementieren

- Bevor die Methode implementiert wird

IntelliJ bietet dann an, den Methodenkopf zu erzeugen

In mehreren Schritten kann bei der Erzeugung eingegriffen werden

- Rückgabetyp bestimmen
- Parametertypen und Parameternamen bestimmen

Refactoring

Manuelle oder automatisierte **Strukturverbesserung** von Quellcode unter **Beibehaltung des beobachtbaren Programmverhaltens**

Soll Lesbarkeit, Verständlichkeit, Wartbarkeit und Erweiterbarkeit verbessern

Manuelles Refactoring

Unterstützung in der IDE (Live-Demo in IntelliJ)

- Rechtsklick -> Refactor

Beispiele für manuelles Refactoring

- Umbenennen von Variablen (Refactor -> Rename)
- Extrahieren von Methoden (Refactor -> Extract Method)

Automatisches Refactoring mit KI (Beispiel)

```java
import java.util.Scanner;

public class GuessingGame {

    public static void main(String[] args) {
        Scanner myScanner = new Scanner(System.in);
        int number = (int) (Math.random() * 100) + 1;
        int guess = 0;
        int trial = 1;
        String message = "Which number from the range 1 to 100 was chosen?";
        while (guess != number) {
            System.out.println(message);
            System.out.println("Try#: " + trial);
            if (myScanner.hasNext()) {
                if (myScanner.hasNextInt()) {
                    guess = myScanner.nextInt();
                    if (guess < number) {
                        message = guess + " is too small.";
                    } else if (guess > number) {
                        message = guess + " is too large.";
                    } else {
                        message = guess + " Congratulation!";
                    }
                } else {
                    message = myScanner.next() + " is not a number!";
                }
            } else {
                message = "Number " + number + " not found.";
                guess = number; // Alternativ hier break;
            }
            trial = trial + 1;
        }
        System.out.println(message);
    }

}
```

KI-Vorschlag

```java
import java.util.Scanner;

public class GuessingGame {
    private static final Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        int number = generateRandomNumber();
        int guess = 0;
        int trial = 1;
        String message = "Which number from the range 1 to 100 was chosen?";

        while (guess != number) {
            System.out.println(message);
            System.out.println("Try#: " + trial);
            guess = getUserGuess();
            message = updateMessage(guess, number);
            trial++;
        }

        System.out.println(message);
    }

    private static int generateRandomNumber() {
        return (int) (Math.random() * 100) + 1;
    }

    private static int getUserGuess() {
        int guess = 0;
        if (scanner.hasNext()) {
            if (scanner.hasNextInt()) {
                guess = scanner.nextInt();
            } else {
                System.out.println(scanner.next() + " is not a number!");
            }
        }
        return guess;
    }

    private static String updateMessage(int guess, int number) {
        if (guess < number) {
            return guess + " is too small.";
        } else if (guess > number) {
            return guess + " is too large.";
        } else {
            return guess + " Congratulation!";
        }
    }
}
```

Debugging von Methoden (1)

```
public class MethodTest4 {

    3 usages
    private static void printLine(int length) {  length: 20
        for (int i = 0; i < length; i++) {  length: 20
            System.out.print("-");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        printLine(10);
        System.out.println("Hello World!");
        printLine(15);
        System.out.println("Hello EP1!");
        printLine(20);
    }
```

Debug: MethodTest4

"main"@1 in group "main": RUNNING

printLine:4, MethodTest4

main:15, MethodTest4

length = 20

In Methode springen (Step into)

Methode verlassen (Step out)

Debugging von Methoden (2)

Beispiele für weitere Möglichkeiten

- Smart step into
- Run to cursor
- Force step into

Details siehe

- https://www.jetbrains.com/help/idea/stepping-through-the-program.html